

SLIIT
SE3082 – Parallel Computing



REPORT
LAB 05

Akash D
IT24101853

Exercise 05

Code Implementation

```
GNU nano 9.1 lab05e.c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int fib(int n) {
    int i, j;
    if (n < 2)
        return n;
    else {
        // Use tasks for parallelizing the recursive calls
        #pragma omp task shared(i) firstprivate(n)
        i = fib(n - 1);

        #pragma omp task shared(j) firstprivate(n)
        j = fib(n - 2);

        // Wait for both tasks to complete before summing
        #pragma omp taskwait

        return i + j;
    }
}

int main() {
    int n = 10; // You can change this to a higher number like 30 or 35 to see performance scaling
    int result;

    double tstart = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        {
            result = fib(n);
        }
    }

    double tstop = omp_get_wtime();

    printf("Fibonacci of %d is %d\n", n, result);
    printf("Execution Time with OpenMP Tasks: %.4f ms\n", (tstop - tstart) * 1000.0);

    return 0;
}
```

Output

```
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise05
$ ./lab05e.o
Fibonacci of 10 is 55
Execution Time with OpenMP Tasks: 3.0000 ms

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise05
$ |
```

Task Parallelization in Fibonacci

Recursive Task Creation (`#pragma omp task`): Each time `fib()` is called, it spawns two independent tasks ($i = \text{fib}(n-1)$ and $j = \text{fib}(n-2)$). These tasks are placed in the OpenMP runtime task pool to be picked up and executed by available threads.

Single Entry Point (`#pragma omp single`): The parallel region is started once, and `#pragma omp single` ensures that the initial call to `fib(n)` is invoked by only one thread, which then spawns the recursive task tree.

Synchronization (`#pragma omp taskwait`): This directive is critical. It forces the current thread to pause and wait until all child tasks spawned within the current block (i and j) have finished computing before allowing the code to return $i + j$.